Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 04

Desarrollo de Software

DOCENTE:
Bestard Aroche, Yunior

CURSO:

Desarrollo de Aplicaciones Empresariales
4 - C24 - Sección CD

DESARROLLO DE APLICACIONES EMPRESARIALES:

Capacidades

1. **Analizar reglas de negocio para identificar entidades, relaciones y cardinalidades** en un modelo de datos.
2. **Implementar relaciones uno a uno, uno a muchos y muchos a muchos mediante el ORM de Django**, incluyendo el uso de un modelo intermedio (through) cuando la relación lo requiera.
3. **Consultar y presentar datos relacionados utilizando Model, View y Template**, aplicando select_related() y prefetch_related() cuando corresponda.

I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

II. Fundamento teórico

- Revise el material de la Semana 4 (relaciones entre modelos en Django: OneToOneField, ForeignKey y ManyToManyField con through) antes del desarrollo del laboratorio.

III. Normas empleadas

- No aplica

IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Python 3.10 o superior, Django 5, un editor de código (Visual Studio Code) y una cuenta de GitHub para el control de versiones.
- La aplicación desarrollada en el laboratorio de la Semana 3 (Models, migraciones, ORM y SQLite) y, cuando corresponda, la investigación propia realizada en la Parte 2 de dicho laboratorio.

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VI. Modo de entrega

- El laboratorio se entrega mediante este mismo documento Word, completando cada ejercicio con su respuesta y la justificación solicitada.
- Cada ejercicio que lo requiera debe incluir captura de pantalla del código de las entidades definidas en models.py, mostrando explícitamente las relaciones implementadas (OneToOneField, ForeignKey y ManyToManyField con through).
- El documento debe incluir la dirección (URL) del repositorio de GitHub donde se aloja el proyecto actualizado.
- El documento final se sube por el medio indicado por el docente (aula virtual / plataforma del curso).

VII. Procedimiento

En la Semana 3 se incorporó persistencia a la aplicación mediante Django Models, migraciones, Django ORM y SQLite, reemplazando los datos temporales en memoria utilizados en la Semana En esta sesión se avanza hacia modelos relacionados: se incorporan los tres tipos de relación que ofrece Django (OneToOneField, ForeignKey y ManyToManyField, esta última mediante un modelo intermedio con through) sobre proyectos ya persistentes.

El laboratorio se desarrolla en dos partes. En la primera, cada estudiante incorpora sobre la aplicación de la Semana 3 los tres tipos de relación vistos en la teoría de la Semana 4. En la

segunda parte, cada estudiante amplía la base de datos de su investigación propia (la desarrollada en la Parte 2 del laboratorio de la Semana 3), agregando las relaciones necesarias y cumpliendo obligatoriamente los criterios que se detallan en esa sección antes de continuar.

El objetivo es que el estudiante comprenda cómo Django representa la cardinalidad entre entidades mediante campos del Model, y cómo esa relación recorre todo el flujo MVT: la relación se define en el Model, la View la consulta mediante el ORM y el Template presenta los datos relacionados.

Request

→ URL

→ View

→ Model (con relaciones)

→ Manager / QuerySet

→ Django ORM

→ SQLite

→ Template

→ Response

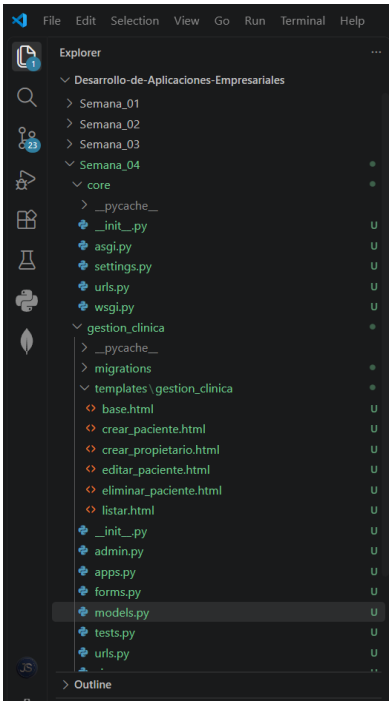
Al finalizar el laboratorio, el estudiante deberá ser capaz de implementar relaciones uno a uno, uno a muchos y muchos a muchos mediante Django ORM, incluyendo un modelo intermedio cuando la relación lo requiera, y de consultar y presentar datos relacionados utilizando select_related(), prefetch_related() y Templates que recorran dichas relaciones.

Enunciados

PARTE 1 — Incorporar las relaciones de la Semana 4 en la aplicación de la Semana 3

Ejercicio 1 — Recuperar la aplicación de la Semana 3

Abra la aplicación desarrollada en el laboratorio de la Semana 3 e identifique su entidad principal (Model), sus Views, sus Templates y el estado actual de sus migraciones. Esta entidad será el punto de partida para las tres relaciones de esta sesión.



Ejercicio 2 — Agregar una relación uno a uno

Diseñe una nueva entidad que funcione como "ficha complementaria" de su entidad principal: información que solo existe si existe el registro principal, y como máximo una por registro. Impleméntela con OneToOneField, defina related_name y justifique el on_delete elegido.



Ejercicio 3 — Agregar una relación uno a muchos

Diseñe una nueva entidad relacionada con su entidad principal mediante ForeignKey. Identifique cuál de las dos entidades representa el lado "muchos" y explique por qué la ForeignKey se declara ahí. Configure related_name y justifique on_delete.



Ejercicio 4 — Agregar una relación muchos a muchos con modelo intermedio

Identifique dos entidades de su proyecto (pueden ser la principal y una nueva) que se relacionen en muchos a muchos y que la relación necesite guardar información propia (mínimo dos campos, por ejemplo cantidad, fecha o estado). Implemente el modelo intermedio y configure ManyToManyField(..., through="...").



Ejercicio 5 — Migrar la nueva estructura

Genere y aplique las migraciones de las tres relaciones nuevas. Documente mediante capturas el resultado de makemigrations, migrate y showmigrations.

```
    + Create model CitatimeWarning: Model 'gestion_clinica.servicio' was already registered. Reloading models is not advised as it can lead to incon
    + Create model HistorialMedicoed models.
    + Create model PacienteServicioel(new_class._meta.app_label, new_class)
    + Add field pacientes_atendidos to servicioissues:
PS C:\Users\BossC\OneDrive\Desktop\Desarrollo-de-Aplicaciones-Empresariales\Semana_04> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, gestion_clinica, sessionsation with model 'Servicio', which is either not installed, or is abstra
Running migrations:
  Applying gestion_clinica.0002_cita_historialmedico_pacienteservicio_and_more... OK model by 'gestion_clinica.Servicio.pacientes_atendidos', but it
PS C:\Users\BossC\OneDrive\Desktop\Desarrollo-de-Aplicaciones-Empresariales\Semana_04> python manage.py showmigrations
admin\Users\BossC\OneDrive\Desktop\Desarrollo-de-Aplicaciones-Empresariales\Semana_04>
 [X] 0001_initial
 [X] 0002_logentry_remove_auto_add
 [X] 0003_logentry_add_action_flag_choices
auth
 [X] 0001_initial
 [X] 0002_alter_permission_name_max_length
 [X] 0003_alter_user_email_max_length
 [X] 0004_alter_user_username_opts
 [X] 0005_alter_user_last_login_null
 [X] 0006_require_contenttypes_0002
 [X] 0007_alter_validators_add_error_messages
 [X] 0008_alter_user_username_max_length
 [X] 0009_alter_user_last_name_max_length
 [X] 0010_alter_group_name_max_length
 [X] 0011_update_proxy_permissions
 [X] 0012_alter_user_first_name_max_length
contenttypes
 [X] 0001_initial
 [X] 0002_remove_content_type_name
gestion_clinica
 [X] 0001_initial
 [X] 0002_cita_historialmedico_pacienteservicio_and_more
sessions
 [X] 0001_initial
```

Ejercicio 6 — Consultar las relaciones desde la View

Implemente al menos una vista que use select_related() para la relación 1:1 o 1:N, y otra que use prefetch_related() para recorrer el modelo intermedio.

.gitignore | models.py U | views.py U

Semana_04 > gestion_clinica > views.py > reporte_relaciones

```python
    return render(request, 'gestion_clinica/eliminar_paciente.html', {'paciente': paciente})

def reporte_relaciones(request):
    pacientes = Paciente.objects.select_related('propietario', 'historial').prefetch_related(
        'citas',
        'pacienteservicio_set__servicio'
    ).all()

    return render(request, 'gestion_clinica/reporte_relaciones.html', {'pacientes': pacientes})
```

Ejercicio 7 — Mostrar los datos relacionados en el Template

Actualice los Templates para mostrar: acceso directo (objeto.relacionado), acceso inverso (objeto.relacionado_set.all() o el related_name definido) y el recorrido del modelo intermedio.

```
                    <td>
                        <strong>Alergias:</strong> {{ pac.historial.alergias|default:"Ninguna" }}<br>
                        <strong>Citas:</strong>
                        <ul class="mb-1 ps-3">
                            {% for cita in pac.citas.all %}
                                <li>{{ cita.fecha_hora|date:"d/m/Y" }}</li>
                            {% endfor %}
                        </ul>
                        <strong>Servicios:</strong>
                        <ul class="mb-0 ps-3">
                            {% for ps in pac.pacienteservicio_set.all %}
                                <li>{{ ps.servicio.nombre }} ({{ ps.estado }})</li>
                            {% endfor %}
                        </ul>
                    </td>
```

Ejercicio 8 — Documentar el flujo

Para una de las tres relaciones implementadas, documente el recorrido completo: Request → URL → View → Model/ORM → SQLite → View → Context → Template → Response, señalando en qué punto interviene la relación. Identifique además qué operación SQL representa conceptualmente cada acción realizada mediante ORM.

Elegí la relación 1:N (Paciente y Citas) para documentar el ciclo completo dentro de la arquitectura MVT:

- **Request:** El usuario intenta acceder al panel de control de la clínica ingresando la ruta en su navegador.
- **URL:** El archivo urls.py detecta esa dirección y llama a la vista correspondiente para listar los pacientes.
- **View (Inicio):** La función en views.py toma el control para procesar la petición.
- **Model/ORM:** Aquí interviene la relación**.** Utilizo el ORM llamando a Paciente.objects.prefetch_related('citas').all().
- **SQLite:** Conceptualmente, esto representa dos operaciones SQL: primero un SELECT * FROM paciente para obtener la tabla principal, seguido de un SELECT * FROM cita WHERE paciente_id IN (...) para extraer solo las citas que le pertenecen a los pacientes consultados.
- **View (Retorno):** La vista recibe estos datos en crudo desde la base de datos y los vincula automáticamente en memoria gracias al ORM.
- **Context:** Empaqueto estos objetos relacionados en un diccionario de contexto (ej. {'pacientes': pacientes}) para enviarlos a la interfaz.
- **Template:** El motor de plantillas lee listar.html. Al ejecutar el bucle {% for cita in paciente.citas.all %}, el template extrae la información ya cruzada desde el contexto y arma la lista visual sin hacer nuevas consultas a la base de datos.
- **Response:** Django entrega el archivo HTML final y renderizado de vuelta al navegador del usuario.

PARTE 2 — Ampliación de la base de datos de la investigación propia (Semana 3)

A partir de aquí trabajará sobre la problemática que investigó en la Parte 2 del laboratorio de la Semana 3 (las cinco entidades ya definidas, con su relación ForeignKey). El objetivo es ampliar ese mismo modelo de datos incorporando los tres tipos de relación, cumpliendo obligatoriamente los criterios de la siguiente tabla antes de continuar con la implementación.

Ejercicio 9 — Auditar el modelo de datos de la Semana 3

Revise su modelo de datos de la Semana 3 frente a los criterios de la tabla siguiente. Documente cuáles cumple directamente, cuáles requieren extender el dominio (agregar una entidad razonable) y cuáles no aplican, con la justificación correspondiente.

Criterio de la Tabla	Estado de Cumplimiento y Justificación
1. Misma problemática y 5 entidades base	Cumple directamente. Se mantiene el dominio de la Clínica Veterinaria con las 5 entidades definidas en la Semana 3: Medicamento, Servicio, Veterinario, Propietario y Paciente.
2. Relación ForeignKey mantenida	Cumple directamente. La relación 1:N se conserva intacta: la entidad dependiente Paciente utiliza una clave foránea que apunta a Propietario, configurada con on_delete=CASCADE y su respectivo related_name.
3. Agregar OneToOneField (perfil/extensión)	Requiere extender el dominio. Se agregará la entidad HistorialMedico vinculada 1:1 a Paciente. *Justificación:* Estos datos clínicos (alergias, peso exacto, notas) son una extensión de la mascota; si los colocáramos como simples campos en Paciente, saturaríamos la tabla principal con información que recepción no necesita ver al momento del registro básico.
4. Agregar ManyToManyField con modelo intermedio (through)	Requiere extender el dominio. Se creará una relación N:M entre Paciente y Servicio. El modelo intermedio PacienteServicio guardará atributos exclusivos de esa transacción, específicamente la fecha_realizacion y el estado de la atención. Estos datos no le pertenecen ni al paciente genérico ni al catálogo de servicios.
5. Justificación de negocio explícita (1:1, 1:N, N:M)	Cumple directamente. • 1:N: Un propietario (dueño) puede registrar y ser responsable de varias mascotas. • 1:1: Cada mascota requiere un único expediente médico consolidado. • N:M: Una mascota a lo largo de su vida recibe múltiples servicios clínicos, y un mismo servicio (ej. vacuna) se aplica a múltiples mascotas.
6. Mínimo siete entidades en total	Cumple directamente. El modelo final consta de las 5 entidades originales + 1 entidad 1:1 (HistorialMedico) + 1 entidad intermedia N:M (PacienteServicio), sumando exactamente 7 entidades.
7. Migraciones generadas y aplicadas	Aplica para la implementación. Se documentará con capturas de pantalla de la terminal (makemigrations, migrate, showmigrations) en el siguiente paso.
8. Vistas y templates con datos relacionados	Aplica para la implementación. Se construirán las vistas utilizando el ORM de Django para mostrar la información cruzada en el dashboard clínico.
9. CRUD del modelo intermedio	Aplica para la implementación. Se creará la funcionalidad para que el personal registre las nuevas atenciones (PacienteServicio) desde el sistema.

Al revisar mi proyecto de la clínica veterinaria de la semana pasada, me di cuenta de que ya cumplía con tener 5 entidades base y la relación 1:N (un Propietario tiene varios Pacientes). Para adaptarlo a los nuevos requisitos, decidí agregar la tabla HistorialMedico como una relación 1:1 con el paciente, ya que son datos clínicos específicos que no quería mezclar en la tabla principal. Además, para la relación N:M, decidí cruzar Paciente con Servicio, creando la tabla intermedia PacienteServicio para registrar la fecha y el estado de cada atención médica.

Criterios a cumplir — Parte 2

Criterio	Verificación
1. Parte de la misma problemática y las mismas cinco entidades investigadas en la Semana 3 (no se cambia de dominio).	Comparar con el Ejercicio 9 del laboratorio de la Semana 3.
2. La relación ForeignKey ya construida se mantiene, con related_name explícito y on_delete justificado por escrito.	Código + justificación en texto.
3. Se agrega como mínimo una relación OneToOneField: una entidad nueva que actúa como ficha/perfil/extensión de una entidad ya existente.	¿Por qué esta entidad no es simplemente más campos de la entidad principal?
4. Se agrega como mínimo una relación ManyToManyField con modelo intermedio (through), y esa relación debe guardar como mínimo dos atributos propios que no pertenecen a ninguna de las dos entidades relacionadas.	¿Qué dato pertenece a la relación en sí, y no a ninguna de las dos entidades? Si no hay respuesta clara, el criterio no se cumple.
5. Cada una de las tres relaciones (1:1, 1:N, N:M) tiene una justificación de negocio explícita, no solo técnica.	Un párrafo por relación.
6. El modelo final tiene como mínimo siete entidades: las cinco originales, la entidad del 1:1 y el modelo intermedio del N:M.	Conteo directo.
7. Las migraciones de las nuevas entidades están generadas y aplicadas, con showmigrations documentado.	Capturas de pantalla.
8. Existe al menos una vista y un template que muestren datos relacionados, no solo el listado plano de una entidad.	Revisión de Views y Templates.
9. El modelo intermedio tiene su propio CRUD (crear, editar y eliminar la relación con sus atributos, no solo asociar identificadores).	Revisión funcional.

Ejercicio 10 — Ampliar el modelo de datos

Diseñe e implemente en models.py las nuevas entidades necesarias para cumplir los criterios 3 y 4 de la tabla anterior: la entidad del 1:1 y el modelo intermedio del N:M. Incluya capturas de pantalla del código de todas las entidades, mostrando claramente las relaciones (OneToOneField, ForeignKey y ManyToManyField con through).

```python
class HistorialMedico(models.Model):
    paciente = models.OneToOneField(Paciente, on_delete=models.CASCADE, related_name='historial')
    alergias = models.TextField(blank=True, null=True)
    peso_actual = models.DecimalField(max_digits=5, decimal_places=2, default=0.00)

    def __str__(self):
        return f"Historial de {self.paciente.nombre}"
```

```python
class Servicio(models.Model):
    nombre = models.CharField(max_length=100)
    duracion_minutos = models.IntegerField()
    precio = models.DecimalField(max_digits=6, decimal_places=2)
    pacientes_atendidos = models.ManyToManyField(Paciente, through='PacienteServicio', related_name='servicios_recibidos')

    def __str__(self):
        return f"{self.nombre} - S/. {self.precio}"
```

Con la lógica definida, me fui al archivo models.py para plasmarlo en código. Para el historial médico utilicé OneToOneField apuntando a Paciente. Para los servicios, agregué un ManyToManyField en la clase Servicio, pero le añadí el parámetro through='PacienteServicio' para obligar a Django a usar mi tabla intermedia personalizada, la cual guarda atributos propios como el estado de la cita. A todas las relaciones les puse on_delete=models.CASCADE para mantener la integridad referencial.

Ejercicio 11 — Migrar la base de datos ampliada

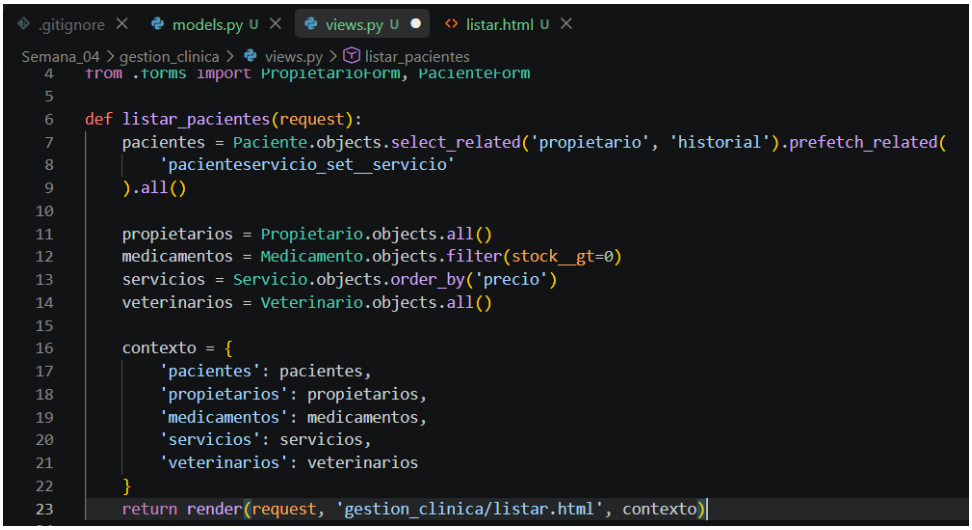
Genere y aplique las migraciones correspondientes a las nuevas entidades y relaciones. Verifique su estado mediante showmigrations.

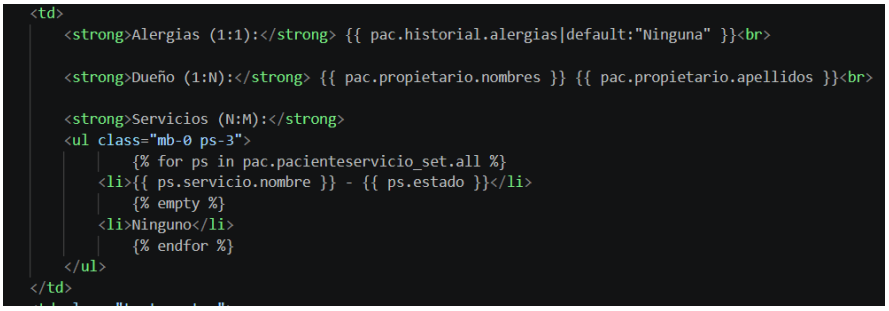
```
PS C:\Users\BossC\OneDrive\Desktop\Desarrollo-de-Aplicaciones-Empresariales\Semana_04> python manage.py makemigrations
Migrations for 'gestion_clinica':
  gestion_clinica\migrations\0003_delete_cita.py
    - Delete model Cita
PS C:\Users\BossC\OneDrive\Desktop\Desarrollo-de-Aplicaciones-Empresariales\Semana_04> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, gestion_clinica, sessions
Running migrations:
  Applying gestion_clinica.0003_delete_cita... OK
PS C:\Users\BossC\OneDrive\Desktop\Desarrollo-de-Aplicaciones-Empresariales\Semana_04> python manage.py showmigrations
admin
 [X] 0001_initial
 [X] 0002_logentry_remove_auto_add
 [X] 0003_logentry_add_action_flag_choices
auth
 [X] 0001_initial
 [X] 0002_alter_permission_name_max_length
 [X] 0003_alter_user_email_max_length
 [X] 0004_alter_user_username_opts
 [X] 0005_alter_user_last_login_null
 [X] 0006_require_contenttypes_0002
 [X] 0007_alter_validators_add_error_messages
 [X] 0008_alter_user_username_max_length
 [X] 0009_alter_user_last_name_max_length
 [X] 0010_alter_group_name_max_length
 [X] 0011_update_proxy_permissions
 [X] 0012_alter_user_first_name_max_length
contenttypes
 [X] 0001_initial
 [X] 0002_remove_content_type_name
gestion_clinica
 [X] 0001_initial
 [X] 0002_cita_historialmedico_pacienteservicio_and_more
 [X] 0003_delete_cita
```

Una vez listo el código, abrí la terminal para sincronizar SQLite. Primero ejecuté python manage.py makemigrations y vi cómo Django detectaba perfectamente las nuevas tablas y relaciones. Luego apliqué los cambios físicos con python manage.py migrate. Para estar cien por ciento seguro, usé showmigrations y comprobé que la nueva migración estaba marcada con una [X]. Todo se integró sin errores.

Ejercicio 12 — Consultar y mostrar las relaciones

Implemente las Views necesarias para consultar los datos relacionados utilizando select_related() y prefetch_related() según corresponda, y actualice los Templates para presentarlos (acceso directo, acceso inverso y recorrido del modelo intermedio).

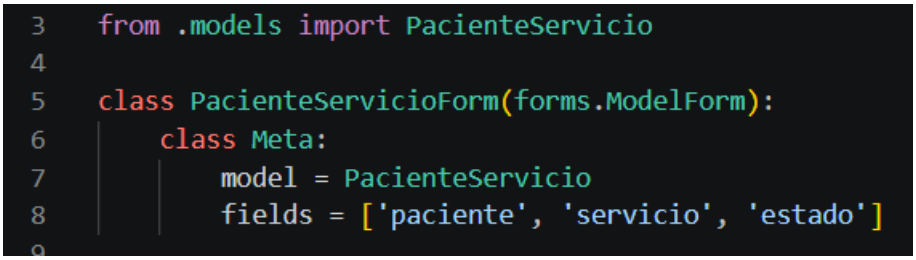




Para que la interfaz mostrara esta nueva información sin matar el rendimiento de la base de datos, optimicé mi vista principal. Usé select_related('propietario', 'historial') para traer los datos 1:1 y 1:N en una sola consulta SQL, y prefetch_related para recolectar todos los servicios (N:M) en memoria. Después, modifiqué el archivo listar.html usando las notaciones de punto (ej. paciente.historial.alergias) y bucles for para mostrar todo organizado en la tabla del panel.

Ejercicio 13 — CRUD del modelo intermedio

Implemente las operaciones de creación, edición y eliminación del modelo intermedio (por ejemplo, agregar, modificar o quitar un elemento de la relación N:M con sus atributos propios).



```python
from django.urls import path
from . import views

urlpatterns = [
    path('', views.listar_pacientes, name='listar_pacientes'),
    path('propietario/nuevo/', views.crear_propietario, name='crear_propietario'),
    path('paciente/nuevo/', views.crear_paciente, name='crear_paciente'),
    path('paciente/editar/<int:id>/', views.editar_paciente, name='editar_paciente'),
    path('paciente/eliminar/<int:id>/', views.eliminar_paciente, name='eliminar_paciente'),
    path('atencion/nueva/', views.registrar_atencion, name='registrar_atencion'),
    path('atencion/editar/<int:id>/', views.editar_atencion, name='editar_atencion'),
    path('atencion/eliminar/<int:id>/', views.eliminar_atencion, name='eliminar_atencion'),
]
```

Necesitaba que el personal de la clínica pudiera registrar las atenciones, así que creé el CRUD para la tabla PacienteServicio. Primero armé su formulario en forms.py. Luego, en views.py, construí las vistas para crear, editar y eliminar registros usando get_object_or_404 para mayor seguridad. Finalmente, conecté estas vistas en urls.py. Ahora es posible asignarle un servicio a un perro, cambiar su estado a "Completado" o cancelar la atención directamente desde la web.

Ejercicio 14 — Publicar en GitHub

Actualicen requirements.txt y README.md describiendo el modelo de datos ampliado (entidades y relaciones). Hagan commit y push al repositorio. Incluyan en este documento la dirección (URL) del repositorio, conforme a lo indicado en Modo de entrega.

```
PS C:\Users\BossC\OneDrive\Desktop\Desarrollo-de-Aplicaciones-Empresariales> git add README.md
>> git commit --amend --no-edit
>> git push origin main --force
[main 45a9e0e] Semana 04: Ampliación del modelo con relaciones 1:1, 1:N y N:M
 Date: Sun Sep 13 23:26:56 2026 -0500
 26 files changed, 847 insertions(+), 14 deletions(-)
 create mode 100644 Semana_04/core/__init__.py
 create mode 100644 Semana_04/core/asgi.py
```



Para cerrar el laboratorio, generé mi archivo requirements.txt usando pip freeze para que cualquier persona pueda instalar las dependencias del proyecto. Actualicé el README.md detallando cómo funciona este nuevo modelo de datos con sus relaciones 1:1, 1:N y N:M. Finalmente, guardé todos los cambios e hice el commit y push hacia la rama main de mi repositorio en GitHub.

https://github.com/JesusCastillo2026/Desarrollo-de-Aplicaciones-Empresariales.git

Entregables

- Cada enunciado debe tener su respuesta, con la justificación y la captura de pantalla solicitada (especialmente el código de los Models con sus relaciones).
- Repositorio en GitHub actualizado, con requirements.txt, README.md y el código correspondiente a cada integrante del equipo.
- Dirección (URL) del repositorio de GitHub incluida explícitamente en este documento, conforme a lo indicado en Modo de entrega.

Conclusiones:

- Evolución del modelo de datos: Logré escalar la base de datos de la clínica veterinaria, pasando de una estructura simple a un modelo relacional completo. Implementar campos OneToOneField y ManyToManyField con una tabla intermedia (through) me enseñó a separar lógicamente la información. Entendí que aislar datos pesados como el historial médico o registrar estados específicos de atenciones en una tabla puente evita saturar las entidades principales.
- Optimización con el ORM: Uno de los mayores aprendizajes fue descubrir que las consultas mal gestionadas pueden matar el rendimiento del sistema. Al aplicar select_related() y prefetch_related() en mis vistas, logré resolver el problema de las múltiples peticiones a la base de datos. Aprendí a obligar a Django a cruzar los datos directamente mediante JOINs en SQLite o a agruparlos en la memoria antes de mandarlos al template.
- Dominio del flujo completo (CRUD): Darle a la tabla intermedia (PacienteServicio) su propia vista y formulario de edición cerró el círculo. Pude ver en la práctica cómo interactúa toda la arquitectura MVT: desde que el usuario hace clic o envía datos en el HTML, hasta que el ORM lo convierte en código SQL seguro y actualiza físicamente el archivo de la base de datos para devolver una interfaz dinámica y persistente.